

Relatório de Laboratório

Laboratório de Experimentação de Software

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	LABORATORIO 01 - Repositórios populares + Setup do Kanban
Grupo (trio)	[Luisa Clara de Paula Lara Silva] · [Matheus Gaston Viana Silveira] · [Mirelly Pego Cordeiro de Alvarenga]
Link do repositório / GitHub Projects	GastonZinhow/lab-experimentacao-software
Data de entrega	26/08/2026

1. Introdução

Repositórios open-source populares exercem grande influência na Engenharia de Software, pois concentram comunidades numerosas, servem como referência técnica para outros projetos e frequentemente sustentam ferramentas usadas no mercado. Apesar disso, a popularidade medida por estrelas não explica sozinha se esses projetos são maduros, recebem contribuições externas, mantêm releases frequentes ou possuem manutenção ativa.

Neste laboratório, o grupo investiga empiricamente os 1000 repositórios com maior número de estrelas no GitHub, buscando caracterizar esses sistemas a partir de métricas extraídas pela API GraphQL. Além da mineração de dados, o Lab01 também inicia o acompanhamento do processo de trabalho do grupo em um GitHub Projects, que será usado nas próximas atividades da disciplina.

1.1 Questões de Pesquisa

As Questões de Pesquisa definidas no enunciado são:

RQ01: Sistemas populares são maduros/antigos?

Métrica: idade do repositório, calculada a partir da data de criação.

RQ02: Sistemas populares recebem muita contribuição externa?

Métrica: total de pull requests aceitas/merged.

RQ03: Sistemas populares lançam releases com frequência?

Métrica: total de releases.

RQ04: Sistemas populares são atualizados com frequência?

Métrica: tempo até a última atualização, calculado a partir da data do último push.

RQ05: Sistemas populares são escritos nas linguagens mais populares?

Métrica: linguagem primária de cada repositório, comparada com uma fonte externa de popularidade de linguagens.

RQ06: Sistemas populares possuem um alto percentual de issues fechadas?

Métrica: razão entre issues fechadas e total de issues.

RQ07: Sistemas escritos em linguagens mais populares recebem mais contribuição externa, lançam mais releases e são atualizados com mais frequência?

1.2 Hipóteses Informais

Antes de observar os resultados completos, o grupo definiu as seguintes hipóteses informais:

RQ01: Repositórios populares tendem a ser maduros, pois precisam de tempo para acumular estrelas, comunidade e reconhecimento.

RQ02: Repositórios populares tendem a receber muitas contribuições externas, principalmente por meio de pull requests.

RQ03: A popularidade não implica necessariamente alta frequência de releases, pois alguns repositórios populares funcionam como listas, materiais de estudo ou coleções de recursos.

RQ04: Repositórios populares tendem a ser atualizados com frequência, ainda que essa frequência dependa do tipo de projeto.

RQ05: Espera-se uma predominância de linguagens populares, como Python, JavaScript, TypeScript, Java, C, C++ e Rust.

RQ06: Espera-se um percentual alto de issues fechadas em projetos com governança ativa, mas com variação significativa entre repositórios.

RQ07: Sistemas escritos em linguagens mais populares tendem a ter mais contribuições externas e atualizações mais frequentes, devido ao tamanho maior de suas comunidades, mas não necessariamente terão mais releases formais. Métrica: resultados das RQ02, RQ03 e RQ04 (% de PRs aceitas, total de releases, dias desde a última atualização) segmentados por grupo de linguagem (popular vs. demais).

1.3 Propostas adicionais do grupo

Como parte dos 30% de inovação, o grupo decidiu complementar a análise do enunciado com as seguintes variáveis e questões derivadas:

Percentual de PRs aceitas: Além do número absoluto de pull requests aceitas, calcular a razão entre PRs aceitas e PRs totais para observar abertura real à contribuição externa.

Ausência de linguagem primária: Registrar repositórios sem linguagem definida, pois listas, guias e coleções podem ser populares mesmo sem serem sistemas tradicionais de software.

Validação amostral por RQ: Conferir manualmente uma amostra de repositórios para comparar os valores coletados com os dados exibidos no GitHub e reduzir risco de erro na coleta.

2. Contexto

Este estudo está inserido no Lab01 da disciplina de Laboratório de Experimentação de Software, sendo a primeira atividade prática do semestre. Por esse motivo, ele tem um papel inicial importante: além de introduzir a mineração de dados em repositórios open-source, também estabelece o processo de trabalho do grupo por meio do GitHub Projects. Esse board Kanban será mantido e evoluído ao longo dos próximos laboratórios, servindo como base para acompanhar tarefas, responsáveis, estados de execução e snapshots de sprint.

O objeto de estudo deste laboratório são os 1.000 repositórios mais populares do GitHub, considerando como critério de popularidade a quantidade de estrelas. A partir desses repositórios, o grupo coleta e analisa métricas relacionadas à idade dos projetos, contribuição externa via pull requests, quantidade de releases, frequência de atualização, linguagem primária e percentual de issues fechadas. Assim, o estudo busca observar se a popularidade de um repositório está associada a características como maturidade, atividade recente, manutenção contínua e participação da comunidade.

A coleta dos dados foi baseada principalmente na documentação oficial da API GraphQL do GitHub, utilizada para estruturar as consultas, entender os campos disponíveis e aplicar paginação na busca pelos repositórios. Também foram consideradas discussões e respostas de funcionários do próprio GitHub em fóruns e canais de suporte, especialmente para esclarecer comportamentos da API, limites de consulta, paginação e interpretação de campos como pull requests, releases, issues e data do último push.

Para a análise da RQ05, relacionada às linguagens mais populares, o grupo adotou o TIOBE Index como referência externa principal. Esse índice foi usado para comparar a linguagem primária de cada repositório com um ranking reconhecido de popularidade de linguagens de programação. A escolha de uma fonte externa é importante porque evita que o grupo defina “linguagem popular” apenas de forma subjetiva, tornando a análise mais rastreável e consistente ao longo do laboratório.

3. Metodologia

3.1 Principais Desafios

Um dos principais desafios técnicos do laboratório foi viabilizar a coleta dos dados pela API GraphQL do GitHub. Para realizar as consultas, foi necessário gerar e utilizar um token de acesso disponibilizado pelo próprio GitHub, pois consultas desse tipo exigem autenticação e estão sujeitas a limites de uso. Mesmo com o token, a coleta dos dados exigiu cuidado com volume, tempo de resposta e estabilidade das requisições.

Outro desafio importante foi a paginação da coleta. O objetivo do Lab01 era analisar os 1.000 repositórios mais populares do GitHub, mas consultar muitos repositórios de uma vez gerava falhas na API. O grupo tentou aumentar o tamanho das páginas para 20 repositórios por requisição, porém ainda encontrou erros do tipo 502 Bad Gateway, relacionados a timeout pelo alto custo da consulta GraphQL. Por isso, a estratégia adotada foi coletar os dados de 10 em 10 repositórios, reduzindo o risco de falhas e tornando o processo mais estável, ainda que mais demorado.

Essa decisão também trouxe um impacto direto no tempo de execução. Como a coleta precisava percorrer 1.000 repositórios em páginas pequenas, foram necessárias várias requisições sucessivas até completar a base. Assim, o grupo precisou equilibrar estabilidade e desempenho: páginas maiores poderiam reduzir o tempo total, mas aumentavam a chance de erro; páginas menores tornavam a coleta mais lenta, porém mais confiável.

Além disso, durante a validação de consistência dos dados, registrada na Issue #8, o grupo identificou uma limitação específica da API do GitHub no campo `releases_count`. A validação foi feita com o script `“lab01/code/src/metrics/validate_data_consistency.py”`, que gerou o relatório `“lab01/code/data/processed/validation_report_1000_repos.md”`. O relatório indicou que não havia duplicidades nem inconsistências lógicas nos 1.000 repositórios, mas confirmou que o campo `releases_count` atingia um teto de 1000 releases imposto pela API do GitHub, afetando 21 dos 1000 repositórios, ou seja, 2,1% da base.

Esse teto foi confirmado com base em uma discussão da comunidade do GitHub, na qual um funcionário do GitHub explica a limitação da API:

<https://github.com/orgs/community/discussions/24566>

Também foram encontrados dados ausentes em `primary_language`: 87 dos 1000 repositórios, correspondendo a 8,7% da base. Esse comportamento foi considerado esperado, pois alguns repositórios populares não representam sistemas de software tradicionais, mas sim listas, guias, materiais de estudo ou coleções de recursos, o que pode fazer com que não exista uma linguagem dominante identificada pelo GitHub.

Por fim, houve um desafio metodológico na validação das métricas coletadas. Como os dados envolvem diferentes conceitos do GitHub, como pull requests, releases, issues, linguagem primária e data do último push, foi necessário conferir a consistência dos campos antes de interpretar os resultados. A validação calculou distribuições básicas para todas as métricas, como mínimo, máximo, média, mediana e desvio, além de identificar outliers pelo método IQR. Esse processo ajudou o grupo a separar problemas reais de coleta, como o teto de releases, de comportamentos esperados da base, como a presença de outliers em estrelas e releases.

3.2 Tomadas de Decisão

Durante o desenvolvimento do Lab01, o grupo precisou tomar algumas decisões metodológicas para tornar a coleta de dados viável, organizada e consistente. Essas decisões envolveram tanto aspectos técnicos da mineração dos repositórios quanto a organização do trabalho no GitHub Projects.

A primeira decisão foi utilizar Python como linguagem principal para os scripts de coleta, exportação e análise dos dados. Essa escolha foi feita porque a linguagem oferece boa produtividade para manipulação de dados, leitura e escrita de arquivos CSV/JSON e tratamento de datas. Além disso, os integrantes do grupo já possuíam familiaridade com Python, o que reduziu o tempo necessário para implementação e facilitou a divisão das tarefas. O principal trade-off dessa escolha foi abrir mão de linguagens com maior desempenho bruto, mas, para o volume de 1.000 repositórios, a produtividade e a simplicidade foram mais importantes que a performance.

Outra decisão importante foi utilizar a API GraphQL do GitHub em vez da API REST. A API GraphQL permitiu consultar, em uma mesma requisição, diferentes campos necessários para as RQs, como data de criação, linguagem primária, releases, pull requests, issues e data do último push. Isso deixou a coleta mais centralizada e reduziu a necessidade de várias chamadas separadas. Por outro lado, consultas GraphQL muito grandes podem ter custo alto e gerar erros ou timeout, o que exigiu cuidado na construção da query e na paginação.

Em relação à paginação, o grupo decidiu coletar os repositórios em páginas de 10 em 10. Inicialmente, foi considerada a possibilidade de aumentar esse número para acelerar a coleta, inclusive com testes usando páginas de 20 repositórios. Porém, essa abordagem gerou erros 502 Bad Gateway, provavelmente por *timeout* ou excesso de carga na consulta. Assim, o grupo optou por uma paginação menor, que tornou o processo mais demorado, mas mais estável e confiável.

O critério de inclusão dos repositórios também foi definido de forma objetiva: foram selecionados os 1.000 repositórios com maior número de estrelas no GitHub. A estrela foi usada como indicador de popularidade por ser uma métrica pública, simples de comparar e diretamente disponível na busca do GitHub. O grupo não filtrou por linguagem, domínio, organização ou tipo de projeto, justamente para preservar a ideia de analisar os repositórios mais populares de forma ampla. Como consequência, a base inclui tanto sistemas de software tradicionais quanto listas, guias e coleções de recursos.

Para a RQ05, o grupo precisou decidir como definir “linguagens mais populares”. Para evitar uma classificação subjetiva, foi adotado o TIOBE Index como referência externa. Essa escolha permite comparar a linguagem primária dos repositórios com um ranking reconhecido de popularidade de linguagens. O trade-off é que o TIOBE mede a popularidade de forma diferente do GitHub, considerando mecanismos de busca e presença geral da linguagem, não

apenas uso em repositórios open-source. Mesmo assim, a fonte foi mantida de forma consistente ao longo da análise.

Também foi decidido registrar separadamente os casos em que o GitHub não informa uma linguagem primária. Esses repositórios não foram removidos da base geral, pois fazem parte dos 1.000 mais populares e ajudam a representar a diversidade do GitHub. Porém, para análises específicas de linguagem, eles foram tratados como casos sem linguagem definida. Essa decisão evita distorcer os percentuais da RQ05 e permite discutir que nem todo repositório popular é necessariamente um sistema escrito em uma linguagem dominante.

No processo de trabalho, o grupo definiu o limite de WIP da coluna Doing/In Progress como 6 issues, permitindo no máximo 2 issues em andamento por integrante. Como o grupo possui três integrantes, esse limite foi considerado adequado para manter todos com tarefas ativas sem acumular trabalho demais em progresso. O trade-off foi equilibrar paralelismo e controle: um limite menor poderia bloquear integrantes em momentos de dependência entre tarefas, enquanto um limite maior poderia gerar excesso de atividades abertas e dificultar o acompanhamento real do progresso.

Por fim, o grupo decidiu usar issues reais do repositório como cartões do GitHub Projects, em vez de tarefas soltas ou draft issues. Essa decisão segue o enunciado do laboratório e melhora a rastreabilidade, pois cada tarefa pode ser associada a responsável, commits, comentários e histórico no repositório. Além disso, o grupo adotou a prática de criar snapshots do board ao final das sprints, já que o GitHub Projects não oferece facilmente um histórico completo de movimentação entre colunas pela API.

3.3 Etapas

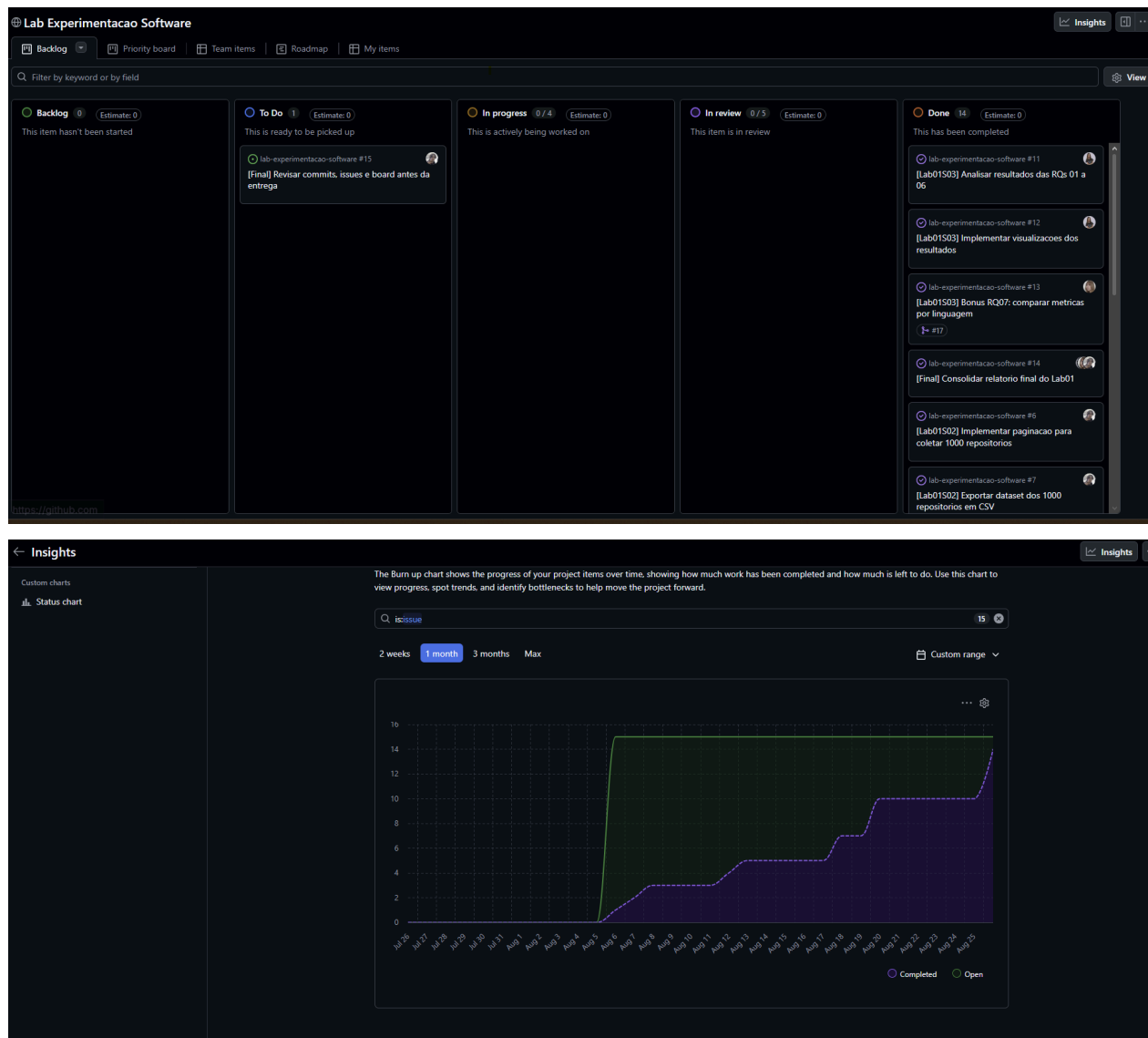
Sprint	Entregas	Responsáveis	Issues(nº)
Lab01S01	Criação do GitHub Projects v2, colunas do board e política de WIP; consulta GraphQL base para 100 repositórios; extração de RQ01/RQ02 (idade, % PRs aceitas), RQ03/RQ04 (releases, última atualização) e RQ05/RQ06 (linguagem primária,	Matheus Gaston (#1, #2, #3); Mirelly Alvarenga (#4); Luisa Silva (#5)	#1, #2, #3, #4, #5

	% issues fechadas)		
Lab01S02	Paginação da coleta para 1.000 repositórios; exportação do dataset em CSV; validação de consistência dos dados (duplicatas, inconsistências lógicas, outliers); primeira versão do relatório com hipóteses informais; script de exportação do snapshot do Project em CSV	Matheus Gaston (#6, #7); Mirelly Alvarenga (#8, #9); Luisa Silva (#10)	#6, #7, #8, #9, #10
Lab01S03	Análise dos resultados das RQ01 a RQ06 e visualização gráfica (histogramas/barras); RQ07 (bônus) — comparação de RQ02/RQ03/RQ04 por linguagem popular vs. demais	Mirelly Alvarenga (#11, #12); Luisa Silva (#13)	#11, #12, #13
Final	Consolidação do relatório final; revisão de commits, issues e board antes da entrega	Matheus Gaston, Mirelly Alvarenga, Luisa Silva (#14); Matheus Gaston (#15)	#14, #15

3.3.1 Configuração do processo

O grupo utiliza o GitHub Projects (v2) vinculado ao repositório, com cartões representando Issues reais (não draft issues), cada uma atribuída a um responsável (campo Assignee). Colunas do board (campo Status): Backlog → To Do → In Progress → Review → Done.

Limite de WIP: 2 issues na coluna In Progress, para um trio de 3 integrantes — permite que cada um mantenha, em média, 1 a 2 tarefas simultâneas em andamento sem sobrecarregar o fluxo do quadro, equilibrando paralelismo com control



3.4 Ferramentas

- **GraphQL API do GitHub:** Mineração dos 1.000 repositórios (busca por estrelas, paginada) e, na Sprint 3, dos itens do GitHub Projects (issue, assignee, status). Consultas e paginação implementadas em script próprio do grupo, sem bibliotecas de terceiros de acesso à API, conforme exigido pelo enunciado.
- **REST API do GitHub:** Usada pontualmente na validação de consistência, para confirmar manualmente, via paginação REST, o teto de 1.000 no campo releases.totalCount da API GraphQL.
- **Python 3.11:** Linguagem de todos os scripts de coleta, exportação e análise.

- **Módulo *statistics* (Python, biblioteca padrão):** Cálculo de mediana, média e desvio padrão em todas as RQs, e de outliers via IQR na validação de consistência.
- **CSV:** Formato de armazenamento do dataset principal e dos snapshots do Project por sprint.
- **GitHub Projects (v2):** Ferramenta de processo obrigatória em todos os laboratórios, com o board do grupo em github.com/users/GastonZinhow/projects/3, vinculado ao repositório [GastonZinhow/lab-experimentacao-software](https://github.com/GastonZinhow/lab-experimentacao-software).

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do repositório	Data atual – data de criação (<i>createdAt</i>), convertida em anos (dias / 365,25)	Anos	Script GraphQL
RQ02	% de pull requests aceitas	<i>merged_pull_requests_count</i> (contagem absoluta)	%	Script GraphQL (<i>pullRequests</i>)
RQ03	Total de releases	<i>releases.totalCount</i>	Contagem	Script GraphQL (<i>releases</i>); sujeito a teto de 1.000 imposto pela API (ver seção 3.1)
RQ04	Dias desde a última atualização	Data atual – data do último push (<i>pushedAt</i>)	Dias	Script GraphQL (<i>pushedAt</i>)
RQ05	Linguagem popular	<i>primaryLanguage.name</i> pertence ao TIOBE top 10 (ago/2026); % calculado sobre repositórios com linguagem definida	% / binário	Script GraphQL (<i>primaryLanguage</i>) + TIOBE Index
RQ06	% de issues fechadas	<i>closed_issues_count</i> / <i>issues_count</i> × 100, calculada apenas para repositórios com ao menos 1 issue	%	Script GraphQL (<i>issues</i>)

RQ07 (bônus)	Comparação de RQ02, RQ03 e RQ04 entre grupos de linguagem	Repositórios separados em dois grupos <i>por primary_language</i> (popular = TIOBE top 10 vs. demais); mediana de % de PRs aceitas, total de releases e dias desde o último push calculada em cada grupo	% / contagem / dias	analyze_rq07.py sobre top_1000_repositories.csv; fonte de popularidade: TIOBE Index
--------------	---	--	---------------------	---

3.6 Inovações Propostas pelo Grupo (30% da nota)

Como parte dos 30% de inovação, o grupo decidiu complementar a análise do enunciado com as seguintes variáveis e questões derivadas:

- Percentual de PRs aceitas: Além do número absoluto de pull requests aceitas, calcular a razão entre PRs aceitas e PRs totais para observar abertura real à contribuição externa.
- Ausência de linguagem primária: Registrar repositórios sem linguagem definida, pois listas, guias e coleções podem ser populares mesmo sem serem sistemas tradicionais de software.
- Validação amostral por RQ: Conferir manualmente uma amostra de repositórios para comparar os valores coletados com os dados exibidos no GitHub e reduzir risco de erro na coleta.

O grupo propôs três inovações (categoria (b): métricas/variáveis adicionais não pedidas no enunciado, e categoria (d): validação metodológica complementar):

1. Percentual de PRs aceitas, além do total absoluto. O enunciado pede apenas o total de pull requests aceitas (RQ02). O grupo optou por calcular também a razão entre PRs aceitas e PRs totais, por repositório, pois o total absoluto favorece repositórios grandes e não diz nada sobre abertura real à contribuição externa — dois repositórios podem ter 500 PRs aceitas, mas um com 90% de aceitação e outro com 20%. Essa métrica alimenta diretamente a RQ02 (seção 4.2) e é a base de comparação usada na RQ07 (seção 4.2).
2. Registro explícito de repositórios sem linguagem primária. Em vez de tratar *primary_language* nulo como dado ausente a ser descartado, o grupo optou por manter esses repositórios na amostra e reportar o percentual separadamente (8,7% da base, seção 4.1) — pois um repositório sem linguagem dominante (listas, guias, materiais de estudo) ainda é um dado relevante sobre o que é popular no GitHub, não um erro de coleta. Essa decisão aparece na metodologia (seção 4.1) e afeta o denominador da RQ05 e da RQ07.

- Validação amostral por RQ antes da integração ao script único. Antes de cada integrante integrar sua extração ao script de coleta do grupo, cada um validou manualmente os campos da própria RQ numa amostra de 5-10 repositórios (issues #3, #4, #5), comparando os valores coletados com o que é exibido diretamente no GitHub. Essa prática reduziu o risco de erro sistemático de coleta antes da paginação para os 1.000 repositórios, e foi complementada por uma validação de consistência mais ampla sobre a base completa (issue #8, seção 4.1: distribuições, outliers via IQR, duplicatas e inconsistências lógicas).

4. Resultados

4.1 Coleta de Dados

A coleta buscou os 1.000 repositórios com maior número de estrelas do GitHub, via *search* da API GraphQL, e os 1.000 foram efetivamente obtidos e analisados, não houve repositório descartado por dado incompleto: todos os campos obrigatórios (nome, dono, URL, data de criação, estrelas, releases, data do último push, pull requests, PRs mescladas, issues, issues fechadas) vieram preenchidos em 100% dos casos. O único campo com lacunas é *primary_language* (nulo em 87 repositórios, 8,7%), tratado como categoria própria e excluído apenas do denominador da RQ05, permanecendo nas demais RQs.

Um ponto analisado da validação que é válido destacar, foi a checagem de outliers, feita via IQR em cada métrica numérica dos 1.000 repositórios.

Métrica	Outliers (IQR)
stars	82
releases_count	92
pull_requests_count	118
merged_pull_requests_count	124
issues_count	100
closed_issues_count	97
age_days	0
days_since_update	194

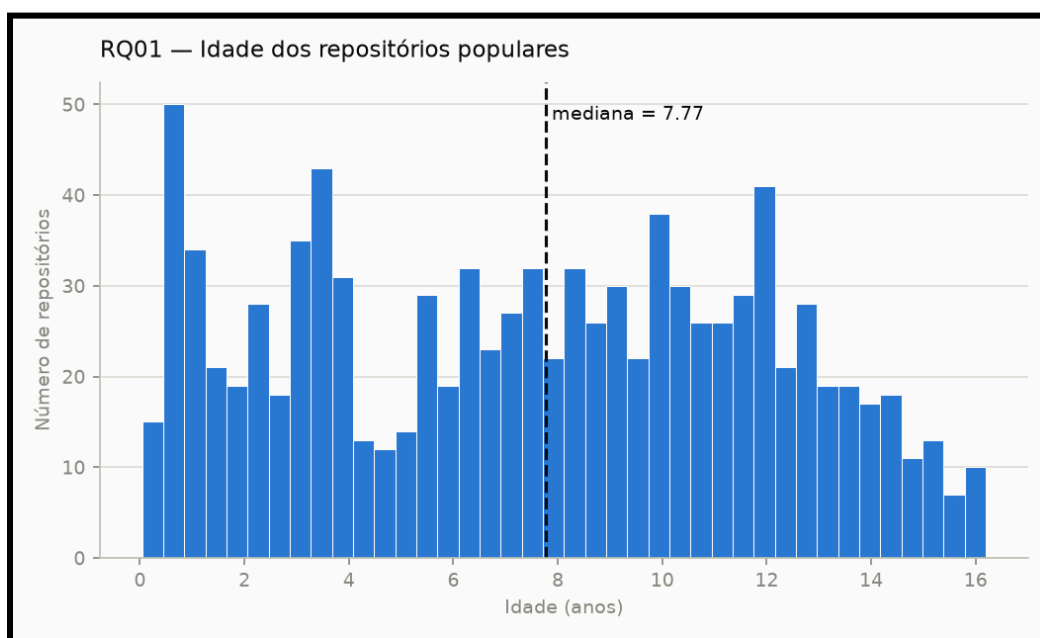
Nenhum outlier foi removido da amostra. A decisão do grupo foi manter todos os valores e reportar mediana em vez de média em todas as RQ, a mediana é pouco sensível a valores extremos, então os outliers não distorcem o número reportado, mas continuam disponíveis para discussão pontual. Por exemplo: *age_days* (idade) não tem nenhum outlier, ou seja, a idade dos repositórios populares é uma distribuição bem mais homogênea que as demais

métricas; já *days_since_update* (dias sem atualização) tem 194 outliers, com casos de até 2.458 dias sem atualização, repositórios "congelados", como listas e materiais de estudo, que não deixam de ser populares por isso.

O único ajuste feito antes da análise foi tratar a saturação em 1.000 do campo *releases_count* (quantidade de releases) que foram 21 repositórios e 2,1% da base como uma limitação de coleta, não como outlier estatístico: a RQ03 é reportada tanto com a base completa quanto excluindo esses 21 casos, para mostrar que a ressalva não muda a mediana.

4.2 Visualização Gráfica

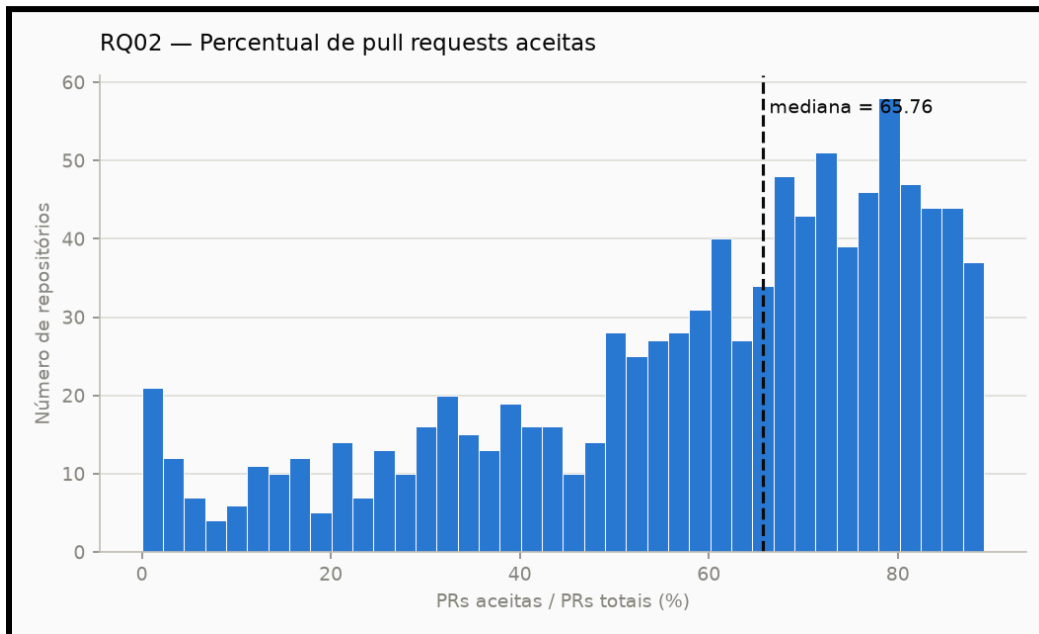
RQ01 - Os sistemas populares são maduros/antigos?



(gráfico: *rq01_idade.png* — histograma)

Mediana: 7,77 anos (média 7,68; mín. 0,04 ano; máx. 18,37 anos; n=1.000).

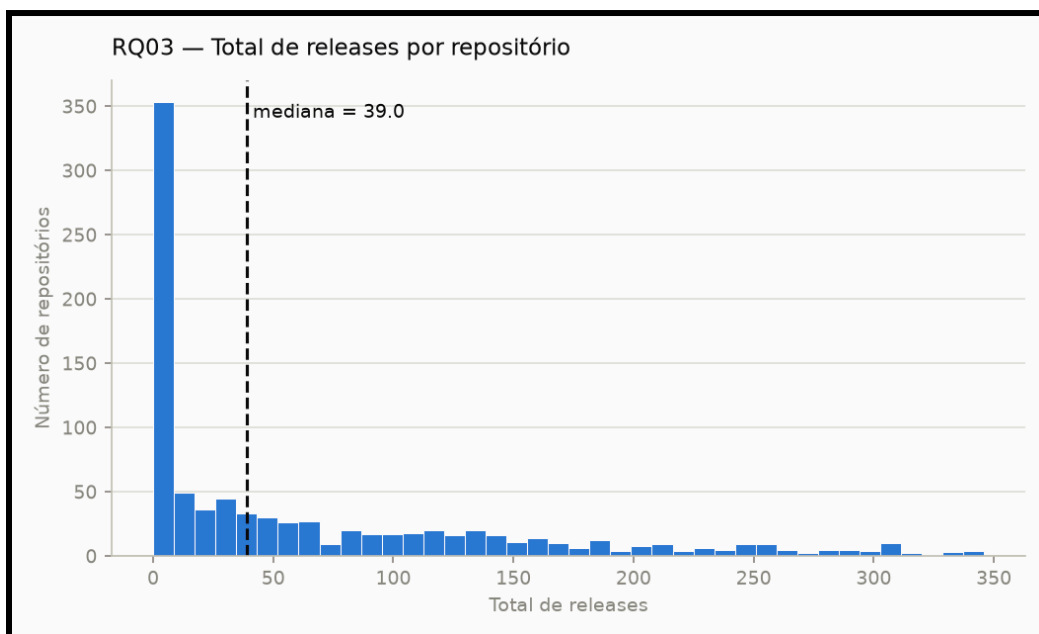
RQ02 - Os sistemas populares recebem muita contribuição externa?



(gráfico: rq02_pr_merge_ratio.png — histograma)

Mediana: 768 pull requests aceitas (média 4.234,14; n=1.000).

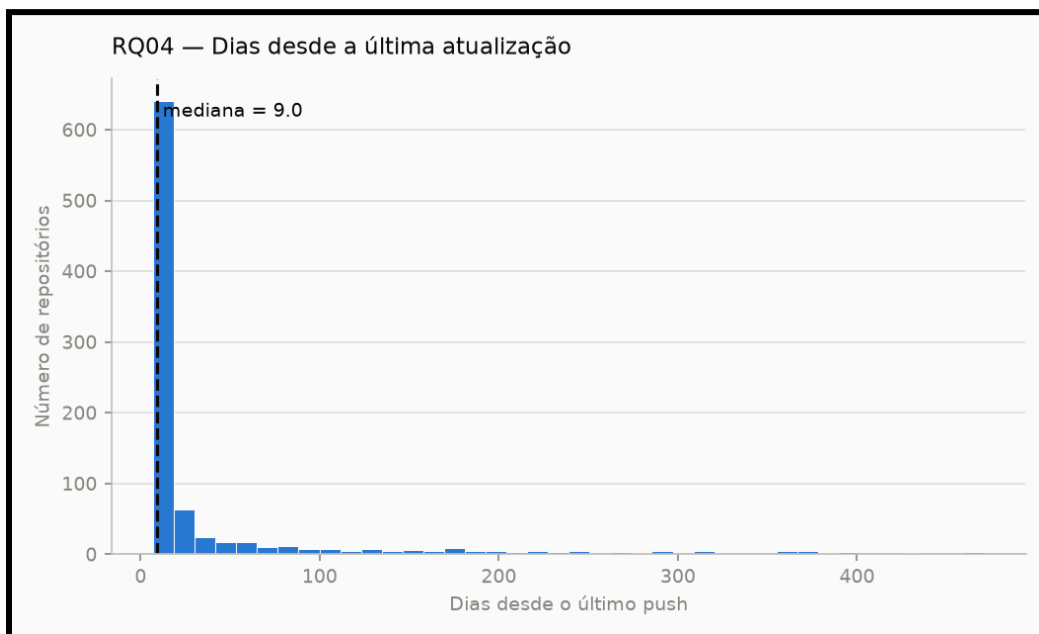
RQ03 — Sistemas populares lançam releases com frequência?



(gráfico: rq03_releases.png — histograma)

Mediana: 39 releases na base completa, 37 excluindo os 21 casos saturados.

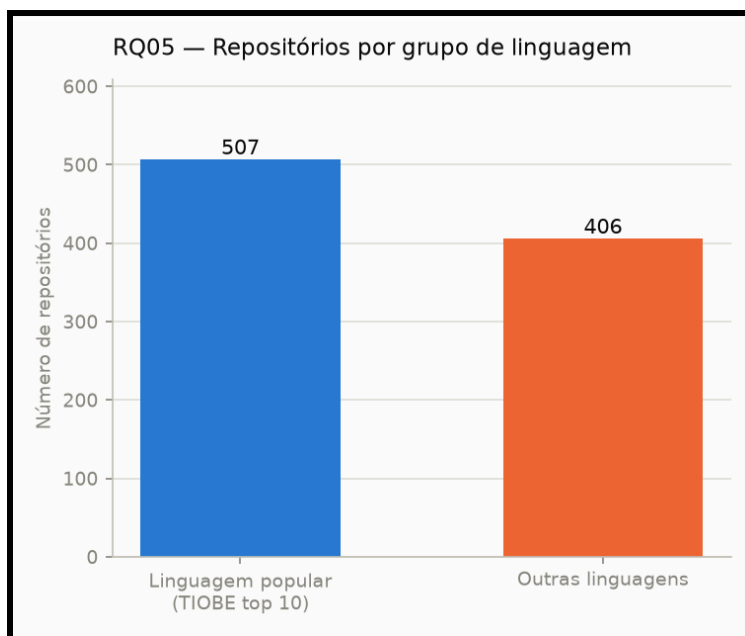
RQ04 — Sistemas populares são atualizados com frequência?



(gráfico: rq04_dias_desde_update.png — histograma)

Mediana: 9 dias (média 120,89; máx. 2.458 dias, mais ou menos 6,7 anos).

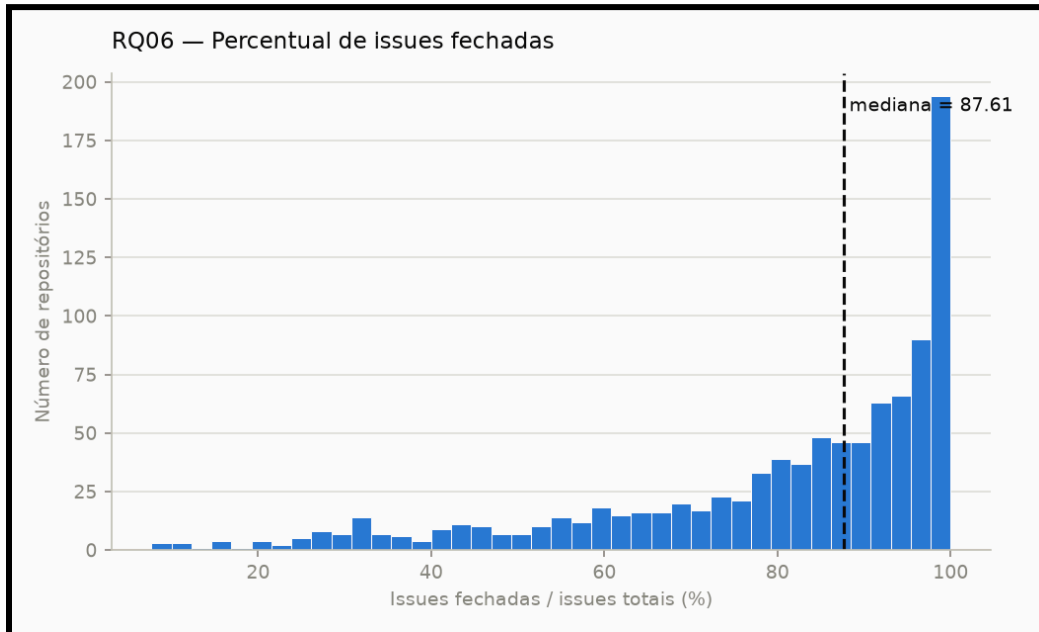
RQ05 — Sistemas populares são escritos nas linguagens mais populares?



(gráfico: rq05_linguagem.png — barras)

55,53% dos repositórios com linguagem definida (507 de 913) usam uma linguagem do TIOBE top 10.

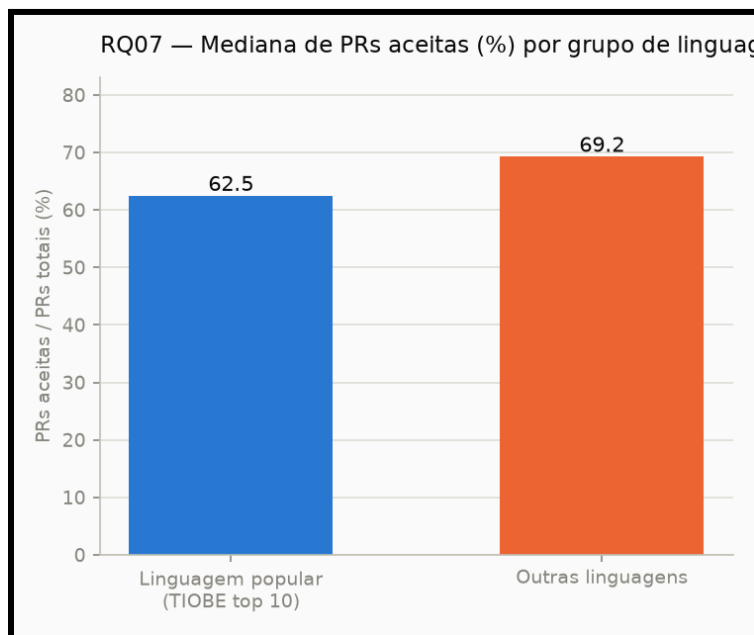
RQ06 — Sistemas populares possuem alto percentual de issues fechadas?



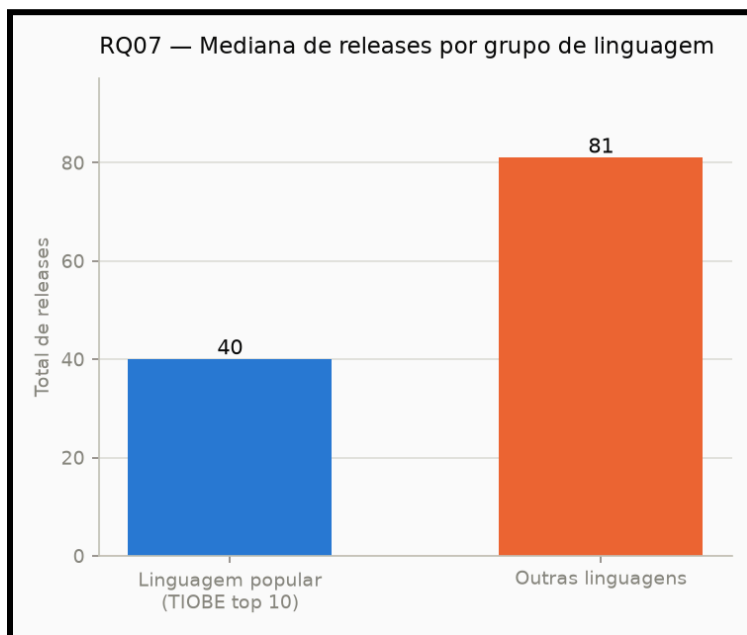
(gráfico: *rq06_issue_close_ratio.png* — histograma)

Mediana: 87,61% (média 80,24%; n=957).

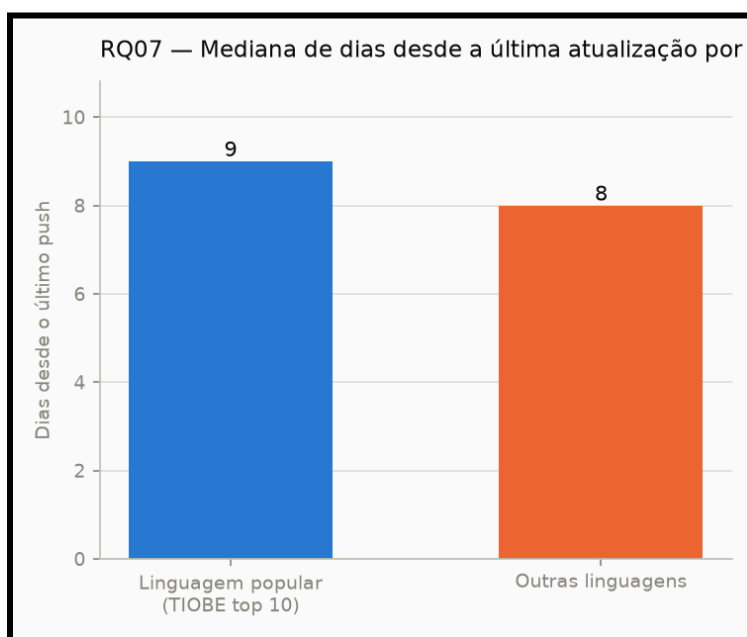
RQ07 — Linguagens populares recebem mais contribuição, lançam mais releases e são atualizadas com mais frequência?



(gráficos: *rq07_pr_merge_ratio.png*)



(gráficos: rq07_releases.png)



(gráficos: rq07_dias_desde_update.png)

RQ02, RQ03 e RQ04 segmentadas por grupo de linguagem (popular = TIOBE top 10, n = 507; outras, n = 406).

Métrica	Mediana — linguagem popular	Mediana — outras linguagens
PRs aceitas (%)	62,45%	69,24%
Releases (total)	40	81

4.3 Discussão

RQ01 (Idade): Hipótese confirmada. A mediana de 7,77 anos confirma que repositórios populares são majoritariamente maduros e a maioria tem entre 4 e 14 anos, consistente com a expectativa de que acumular estrelas exige tempo de exposição. O caso extremo de 0,04 ano (mais ou menos 2 semanas) confirma também a ressalva da hipótese sobre "hype pontual do repositório", mas é exceção, não regra.

RQ02 (Contribuição Externa): Hipótese confirmada. A mediana de 768 pull requests aceitas confirma volume relevante de contribuição externa. A média (4.234,14) bem acima da mediana mostra grande variação entre repositórios, consistente com a hipótese de que o rigor de revisão de cada mantenedor influencia o volume de PRs aceitas.

RQ03 (Releases): Hipótese confirmada, com ressalva metodológica. A mediana de 39 (37 sem os casos saturados) mostra que a maioria dos repositórios populares de fato lança releases, mas a média muito acima da mediana confirma a variação por tipo de projeto prevista na hipótese (poucos projetos com centenas/milhares de releases puxam a média, enquanto a mediana reflete o repositório "típico"). O teto de releases.totalCount em 21 repositórios não muda a conclusão, mas subestima a cauda superior da distribuição e deve ser lido como limitação da API, não do processo de coleta.

RQ04 (Atualização): Hipótese confirmada. A mediana de 9 dias mostra que a grande maioria dos repositórios populares está ativamente mantida. Como previsto na hipótese, existe uma cauda de repositórios "congelados", o máximo de 2.458 dias (mais ou menos 6,7 anos) sem push é consistente com conteúdo estático (listas, materiais de estudo) que continua popular sem precisar de manutenção contínua.

RQ05 (Linguagens Populares): Hipótese confirmada, mas por margem menor do que o esperado. Dos repositórios com linguagem definida, 55,53% (507 de 913) usam uma linguagem do *TIOBE top 10*, que são maioria, mas não a maioria esmagadora que a hipótese antecipava. Quase metade (44,47%) usa linguagens fora do *top 10 do TIOBE*, o que sugere que popularidade de repositório e popularidade de linguagem se reforçam menos do que o previsto.

RQ06 (Issues Fechadas): Hipótese confirmada. A mediana de 87,61% de issues fechadas é alta, confirmando processos de triagem maduros nos repositórios populares. A cauda inferior (mínimo 7,69%) é consistente com a ressalva da hipótese sobre projetos muito grandes, cujo volume de issues supera a capacidade de resposta da manutenção.

RQ07 (linguagem popular vs. demais): Hipótese refutada para PRs aceitas e releases. Ao contrário do esperado, repositórios em linguagens fora do *TIOBE top 10* apresentam mediana maior de PRs aceitas (69,24% vs. 62,45%) e de releases (81 vs. 40) do que os em linguagens populares, ou seja, linguagens populares não apresentam mais contribuição externa nem mais releases; apresentam menos. Uma leitura possível é que parte do grupo "outras linguagens"

inclui ecossistemas com convenções fortes de release e contribuição (ex.: Go, TypeScript, Shell fora do TIOBE top 10, mas comuns em ferramentas de infraestrutura/DevOps altamente colaborativas). Já para dias desde a última atualização, a diferença entre grupos é pequena (9 vs. 8 dias) e não muda a conclusão geral da RQ04.

Uma discussão a ser feita é a ameaça à validade, pois, a amostra é um corte único no tempo, então não captura tendência dos repositórios, apenas o estado atual dos 1.000 repositórios mais populares. Estrelas foram usadas como único proxy de popularidade, sem medir uso real (downloads, dependentes).

5. Conclusão

Os 1.000 repositórios mais populares do GitHub confirmam, para as seis RQs do enunciado, o perfil esperado de um projeto open-source maduro: são majoritariamente antigos, recebem contribuição externa relevante, lançam releases com regularidade, permanecem ativamente mantidos e resolvem a grande maioria das issues abertas. A única nuance real fica na relação entre popularidade do projeto e popularidade da linguagem em que é escrito, e usar uma linguagem do topo do TIOBE é comum, mas está longe de ser unanimidade, o que sugere que esse vínculo é mais fraco do que a intuição inicial do grupo previa.

A RQ07 (bônus) aprofunda exatamente essa nuance e vai além dela: a hipótese do grupo foi refutada. Repositórios em linguagens fora do TIOBE top 10 recebem mais contribuição externa e lançam mais releases do que os em linguagens populares o oposto do esperado. Ou seja, não só o vínculo entre popularidade de linguagem e de projeto é mais fraco do que o previsto (RQ05), como na direção oposta à intuição inicial ele nem existe (RQ07): estar entre as linguagens mais populares do mercado não torna um repositório mais colaborativo ou mais ativo em releases.

A coleta é um corte único no tempo, os resultados descrevem o estado dos repositórios populares num momento específico, não uma tendência. Estrelas foram usadas como único proxy de popularidade, o que captura visibilidade mas não uso real (downloads, dependentes, forks ativos). O tamanho da amostra é definido pelo enunciado e é grande o suficiente para generalizações sobre o topo do ranking, mas não diz nada sobre repositórios fora dele. Uma limitação técnica específica (o teto de contagem da API) foi identificada e discutida, mas não totalmente corrigida, e o grupo optou por reportar a ressalva em vez de recorrer manualmente os casos afetados nesta entrega.

Com mais tempo, o grupo recontaria via paginação REST os repositórios afetados pelo teto de releases, para eliminar de vez essa ressalva em vez de apenas documentá-la, e aprofundaria a RQ07 por linguagem individual em vez de só dois grupos binários para entender quais linguagens específicas puxam o resultado do grupo "outras linguagens" para

cima. Das frentes de inovação propostas pelo grupo, a que mais vale a pena expandir em um trabalho futuro é a validação sistemática de qualidade de dados (detecção de outliers, inconsistências e valores ausentes antes da análise), pois ela já se mostrou útil para identificar um problema real de coleta nesta própria entrega, e generalizá-la como etapa padrão dos próximos laboratórios da disciplina reduziria o risco de reportar métricas distorcidas sem perceber.

6. Referências

TIOBE Index. TIOBE Software, ago. 2026. Fonte usada para definir "linguagens mais populares" nas RQ05 e RQ07, conforme pedido no enunciado do Lab01. Disponível em: <https://www.tiobe.com/tiobe-index/>.

GITHUB. GraphQL API Docs. Documentação oficial da API usada para toda a coleta de dados do laboratório. Disponível em: <https://docs.github.com/en/graphql>.

GITHUB. REST API Docs, Releases. Documentação usada na validação de consistência para confirmar manualmente, via paginação REST, o teto de 1.000 no campo `releases.totalCount` da API GraphQL. Disponível em: <https://docs.github.com/en/rest/releases/releases>.